



p k t

PRODUCTION PLATFORM · VYNE INFRASTRUCTURE

Real-Time NetFlow Visibility & Alerting

**A production-grade network traffic analysis platform built for
multi-site enterprise infrastructure**

```
<div class="cover-stats">
  <div class="stat-item">
    <span class="stat-value">~148</span>
    <span class="stat-label">FLOWS / SECOND</span>
  </div>
  <div class="stat-item">
    <span class="stat-value">2</span>
    <span class="stat-label">COLLECTOR SITES</span>
  </div>
  <div class="stat-item">
    <span class="stat-value">4+</span>
    <span class="stat-label">ROUTER SAMPLERS</span>
  </div>
  <div class="stat-item">
    <span class="stat-value">v26.5</span>
    <span class="stat-label">CLICKHOUSE ENGINE</span>
  </div>
</div>
```

pktFlow v1.0 · Vyne Infrastructure · Confidential

Live at <http://172.23.80.5:8766>
Branch: feature/initial-build



pkt **FLOW**

EXECUTIVE SUMMARY

What is pktFlow?

pktFlow is a production-deployed NetFlow visualization and alerting platform purpose-built for Vyne's multi-site enterprise infrastructure. It collects live NetFlow data from network routers across Medical and Dental sites, stores it in a high-performance ClickHouse columnar database, and serves a rich React dashboard for real-time and historical traffic analysis.

Unlike commercial solutions, pktFlow is fully self-hosted, requires no licensing fees, and is architected to scale with Vyne's infrastructure without vendor lock-in.



Full Traffic Visibility

Every flow in and out of your network is captured, decoded, and stored. See who's talking to whom, on what ports, at what volume — across all sites, in real time.



Proactive Alerting

Rule-based alert engine detects anomalies: data gaps, new hosts appearing on the network, traffic spikes, and unusual port-protocol combinations.



Multi-Site, Multi-Sampler

Aggregates NetFlow from all major routers across QTS, OneNeck, and Dental sites into a single unified view with per-site filtering.



Role-Based Access

Admin, Analyst, and Viewer roles with JWT authentication and bcrypt password security. All access is auditable and site-scoped.

Live Deployment

pktFlow is currently running in production at `http://172.23.80.5:8766` on the O2 server, receiving approximately **148 flows per second** combined from Medical and Dental collector infrastructure.

Business Value

Operational Benefits

- Real-time visibility into network traffic patterns across all sites
- Immediate detection of unusual hosts, data gaps, or traffic anomalies
- Flow-level export (CSV/JSON) for compliance and incident response
- Network topology view showing device relationships via D3 graph
- Historical query and filtering for forensic investigation

Strategic Benefits

- No per-seat or per-flow licensing — fully self-hosted
- Built on open-source components (goflow2, Vector, ClickHouse)

- Extensible architecture — new collector sites add with minimal config
- OIDC-ready (Okta integration scaffolded) for SSO when needed
- Notification channels: Slack, Email, PagerDuty, Webhook (in progress)



pkt **FLOW**

PLATFORM CAPABILITIES

Core Feature Set



Live Dashboard

Real-time metrics: total flows, bytes/packets, top talkers, protocol mix, and per-site breakdowns. Refreshes on demand.



Flow Explorer

Filter flows by src/dst IP, port, protocol, site, and time range. Paginated table view with CSV and JSON export for downstream analysis.



Device View

Complete host inventory showing every device observed on the network, with first/last seen timestamps and flow statistics per device.



Network Topology

Interactive D3 force-directed graph rendering device-to-device communication relationships. Visualizes the actual traffic map.



Alert Engine

Rule-based alerting for data gaps, new host detection, traffic threshold crossing, and port/protocol anomalies. Configurable per-rule.



Notifications

Alert delivery to Slack, Email, PagerDuty, and Webhook endpoints. Configurable per-channel with notification log retention.



Authentication & RBAC

JWT + bcrypt local auth. Three roles: Admin (full access), Analyst (read + export), Viewer (read-only). OIDC scaffolded for Okta.



Settings Console

In-app settings for ingest tokens, notification channels, OIDC config, storage backends, and service restart — no SSH required.



Multi-Site Collection

Independent collector deployments at Medical (172.23.80.11) and Dental (10.56.57.181) sites. Flows tagged with site label for filtering.

Dashboard Pages

Page	Description	Status
Dashboard	Live aggregate metrics, top talkers, protocol mix, per-site stats	LIVE
Flow Explorer	Full flow table with filters, pagination, CSV/JSON export	LIVE
Device View	Host inventory with first/last seen and per-device flow counts	LIVE
Topology	D3 force-directed network graph of device communication	LIVE
Alerts	Alert rules list, alert event log, rule configuration	LIVE
Settings	Ingest tokens, notification channels, OIDC, service restart	LIVE
Users	User management, role assignment, password reset (admin-only)	LIVE
Traffic by Port	Protocol mix chart, top ports by bytes/flows, port inventory table	PLANNED
Sankey Diagram	D3 alluvial flow: src_ip → dst_port → dst_ip, band width by bytes	PLANNED



pkt **FLOW**

DATA FLOW & PIPELINE

How Data Moves

pktFlow uses a two-tier ingest architecture: a remote collector pipeline (recommended, production-proven) and an optional direct UDP listener. The pipeline below describes the production path currently operating at Vyne.

1

Router → Collector (UDP NetFlow v9)

Network routers export NetFlow v9 records via UDP to port 2055 on the designated collector host. Each flow record contains source/destination IPs, ports, protocol, byte/packet counts, timestamps, and interface identifiers. Vyne currently exports from QTS (172.27.28.89) and OneNeck (192.168.44.7/8) to the Medical collector, and from Dental site routers to the Dental collector.

```
<div class="flow-step">
  <div class="step-num">2</div>
  <div class="step-content">
    <h4>goflow2 – Protocol Decode (PascalCase JSON)</h4>
    <p>goflow2 listens on UDP :2055, decodes NetFlow v9 (including template negotiation) and
    IPFIX, and emits one JSON object per flow to stdout. Output uses PascalCase field names
    (<code>SrcAddr</code>, <code>DstAddr</code>, <code>Proto</code> as integer, etc.). goflow2 is
    stateful – it caches NetFlow v9 templates between packets and recovers automatically after
    restart.</p>
  </div>
</div>

<div class="flow-step">
  <div class="step-num">3</div>
  <div class="step-content">
    <h4>Vector – Transform & Forward (snake_case JSON arrays)</h4>
    <p>Vector reads goflow2 output (via stdin pipe or exec source), applies two transforms:
    <code>parse_json</code> (deserializes the message field) and <code>add_site</code> (injects
    <code>site</code> and <code>sampler_address</code> fields based on sampler IP). Vector then
    batches events and POSTs JSON arrays to <code>pktFlow /api/ingest/flows</code> with a Bearer
    token. Output fields are renamed to snake_case (<code>src_addr</code>, <code>dst_addr</code>,
    <code>proto</code> as string: "UDP"/"TCP"/"ICMP").</p>
  </div>
</div>

<div class="flow-step">
  <div class="step-num">4</div>
  <div class="step-content">
    <h4>pktFlow Ingest API – Normalize & Buffer</h4>
    <p>The FastAPI ingest endpoint receives the JSON array, validates the Bearer token, then
    passes each record through the normalizer (<code>normalize_batch()</code>). The normalizer
    converts snake_case fields to typed <code>FlowRecord</code> objects – converting string
    protocol names to IANA integers, handling empty next-hop fields, and tagging with site
    metadata. Normalized records enter the <code>IngestBuffer</code>, which flushes asynchronously
    to ClickHouse in batches.</p>
  </div>
</div>
```

```

    </div>
</div>

<div class="flow-step">
  <div class="step-num">5</div>
  <div class="step-content">
    <h4>ClickHouse – Columnar Storage</h4>
    <p>Flow records are written to the <code>pktflow.flows</code> MergeTree table in ClickHouse at localhost:9000. The ClickHouse client uses a <code>threading.Lock()</code> to serialize writes safely across asyncio threads, with automatic retry-on-failure. ClickHouse's columnar storage and MergeTree engine make aggregation queries (GROUP BY protocol, top talkers, time-series buckets) extremely fast even over large datasets.</p>
  </div>
</div>

<div class="flow-step">
  <div class="step-num">6</div>
  <div class="step-content">
    <h4>Query API → React Dashboard</h4>
    <p>Browser clients authenticate via JWT (Login page) and query <code>/api/flows/*</code> endpoints. The FastAPI query layer runs parameterized ClickHouse queries and returns JSON. The React frontend (Vite-built, served as static files by pktFlow) renders the dashboard, flow explorer, topology graph, and alert views. Role-based access is enforced server-side on every API call.</p>
  </div>
</div>

```

Key Data Transformation: goflow2 → Vector → pktFlow

goflow2 outputs `PascalCase` field names with `Proto` as an integer (6=TCP, 17=UDP). Vector transforms these to `snake_case` with `proto` as a string ("TCP", "UDP"). The pktFlow normalizer handles the reverse conversion — this chain is fully documented in `DATAFLOW.md` and the normalizer source at `app/ingest/normalizer.py`.



pkt **FLOW**

TECHNICAL SPECIFICATIONS
Infrastructure & Deployment

ROLE	HOST / IP	USER	OS	KEY SERVICES
O2 Server — pktFlow + ClickHouse	172.23.80.5	ec2-user	Amazon Linux 2023	pktflow.service · ClickHouse :9000 · Port 8766
Medical Collector	172.23.80.11	ec2-user	Amazon Linux 2023	goflow2-vector.service · :2055 UDP
Dental Collector	10.56.57.181	ec2-user	Amazon Linux 2023	goflow2-vector.service · :2055 UDP

Technology Stack

BACKEND

FastAPIREST API framework (Python)
ClickHouse v26.5Columnar time-series storage
SQLiteUsers, settings, alerts, events
PydanticFlowRecord schema validation
Python asyncioAsync ingest + query handling
threading.LockThread-safe ClickHouse client

FRONTEND

React 18UI framework
ViteBuild tool (built on O2 Linux)
D3.jsNetwork topology + Sankey charts
TypeScriptType-safe component development
RechartsTime-series and stat charts
React RouterClient-side navigation

COLLECTOR PIPELINE

goflow2NetFlow v9 / IPFIX / sFlow decoder
Vector (vectordotdev)Transform + HTTP batch forwarder
systemdProcess supervision
NetFlow v9Wire protocol from routers
TOML configVector pipeline configuration

SECURITY & AUTH

JWT (HS256)Stateless session tokens
bcryptPassword hashing
Bearer tokensIngest API authentication
RBACAdmin / Analyst / Viewer roles
Okta OIDCScaffolded — integration pending

Key Technical Details

Component	Specification
Ingest endpoint	POST /api/ingest/flows · Bearer token · JSON array payload
Ingest rate (current)	~148 flows/second combined (Medical + Dental)

ClickHouse table	<code>pktflow.flows</code> · MergeTree engine · columnar, compressed
Storage concurrency	<code>threading.Lock()</code> serializes all writes — safe under asyncio task concurrency
NetFlow protocol	v9 (template-based) via <code>goflow2</code> · IPFIX also supported
Collector ingest mode	Medical: stdin pipe · Dental: exec source (<code>systemd stdin=/dev/null</code>)
Vector retry behavior	Exponential backoff on HTTP failure · immediate retry on connection reset
Frontend build	Vite + TypeScript · built on Linux /tmp to avoid Windows rollup binary mismatch
Auth token lifetime	Static JWT secret — tokens survive service restarts by design
Git remotes	GitHub (<code>github</code>) + GitLab (<code>gitlab</code>) · branch: <code>feature/initial-build</code>



pkt **FLOW**

ALERT ENGINE & ROADMAP

Alert Engine

The pktFlow alert engine evaluates rules on a configurable schedule and writes events to the `alert_events` SQLite table when conditions are met. Each rule is stored with type, threshold, severity, and notification routing configuration.

Rule Type	Description	Status
data_gap	Fires when no flows are received for N consecutive minutes. Detects collector outages or router export failures.	FUNCTIONAL
new_host	Fires when a previously unseen IP appears in flow data. Useful for detecting unauthorized devices or new infrastructure.	FUNCTIONAL
threshold	Fires when traffic volume (bytes or flows) exceeds a configured ceiling within a sliding window.	ENGINE STUB
rate_spike	Fires when traffic rate increases sharply relative to baseline. DDoS and exfiltration detection.	ENGINE STUB
port_protocol	Fires when traffic on specific port/protocol combinations exceeds threshold or appears for the first time.	ENGINE STUB

Notification Channels

Alert delivery code is written for Slack (webhook), Email (aiosmtplib + Jinja2), PagerDuty, and generic Webhook. Backend code exists in `engine.py` but notification channels have not been tested end-to-end on the O2 deployment. A "Send Test" API endpoint is planned to validate channel configuration.

Development Roadmap

Feature	Description	Priority
Traffic by Port Page	Protocol mix chart, top ports by bytes/flows, full port inventory table, filterable by sampler/site	NEXT UP
Sankey Flow Diagram	D3-sankey alluvial chart: src_ip → dst_port → dst_ip, band width proportional to byte volume	NEXT UP
Alert Rule Completion	Implement threshold, rate_spike, and port_protocol rule execution logic	IN PROGRESS
Topology Drill-Down	Clickable topology nodes that filter Flow Explorer to src/dst flows for that device	PLANNED
Okta OIDC Auth	Complete app/auth/okta.py backend — settings UI and DB column already exist	PLANNED
Topology Export	PNG/SVG/JSON export button for network topology view with optional per-device filtering	PLANNED

Notification Testing	"Send Test" API endpoints for Slack, Email, PagerDuty, and Webhook channels	PLANNED
Alert Event Cleanup	Automated purge job for alert_events and notification_log (SQLite accumulates indefinitely)	PLANNED
Direct UDP Ingest	Complete app/ingest/udp_listener.py for router-direct ingest (no collector host needed)	PLANNED
AI Assistant	Anthropic API-powered traffic analysis assistant — backend + frontend written, needs API key + testing	PLANNED



pkt **FLOW**

QUICK REFERENCE

API & Endpoint Reference

Endpoint	Method	Description
/api/ingest/flows	POST	Receive JSON array of flow records · Bearer token auth
/api/ingest/stats	GET	Ingest buffer stats: buffered, total_received, total_flushed, last_flush
/api/flows/	GET	Paginated flow query with filters (src/dst IP, port, proto, site, time)
/api/flows/summary	GET	Aggregate statistics: total flows, bytes, packets, top talkers
/api/flows/devices	GET	Device inventory with first/last seen and flow counts
/api/flows/topology	GET	Device communication pairs for D3 topology rendering
/api/flows/export	GET	Export flows as CSV or JSON with applied filters
/api/auth/login	POST	JWT auth — returns access token on valid credentials
/api/users/	GET/POST	User management — admin-only
/api/alerts/rules	GET/POST	Alert rule CRUD
/api/alerts/events	GET	Alert event history
/api/settings/	GET/PUT	Platform settings (ingest token, OIDC, notifications)

Service Operations

Action	Command / Path
App directory	/mnt/software/pktflow/
Service status	systemctl status pktflow
Service restart	sudo systemctl restart pktflow
Live URL	http://172.23.80.5:8766
Config file	/mnt/software/pktflow/config.yaml
SQLite DB	/mnt/software/pktflow/pktflow.db
ClickHouse DB	pktflow · user: default · no password
Frontend build	scripts/deploy_frontend.py via Desktop Commander

NetFlow Field Mapping

goflow2 Output	pktFlow Field
SrcAddr	src_addr (string)

DstAddr	dst_addr (string)
SrcPort	src_port (int)
DstPort	dst_port (int)
Proto: 6	proto: "TCP" → stored as 6
Proto: 17	proto: "UDP" → stored as 17
Bytes	bytes (int)
Packets	packets (int)
(Vector added)	site: "oneneck" / "qts" / "dental"

Diagram Assets

Two companion SVG diagrams are included in this documentation package: **pktflow_dataflow.svg** (end-to-end pipeline from router to dashboard with protocol labels) and **pktflow_architecture.svg** (infrastructure layout with server IPs, services, and connection types). Both are scalable vector assets suitable for presentations, Confluence, or print.